

JavaScript 1.1

Execution Model, Declarations & Scope

Print-first interview refresher · fundamentals → traps → answer framework

Ask three questions:

Where is the name?

Is it initialized? What is active?

1 · Name lifecycle: declaration → binding → value

Hoisting is observable pre-execution setup. The engine does not physically move source code.



State after declaration instantiation, before line-by-line execution

function declaration

initialized → function object
an earlier call usually works

var

initialized → undefined
initializer assigns during execution

let · const · class

uninitialized → TDZ
early access → ReferenceError

2 · Scope & name lookup

Scope: where an identifier refers to a particular binding.

Main boundaries

global — outermost host scope

function — params, locals, var

block — let / const / class

module — top level of an ES module

Identifier resolution

inner Environment Record
suffix = "!"



outer Environment Record
source = "outer" (found)



global Environment Record
source = "global"

The first matching binding stops lookup.

A TDZ binding still shadows outer names.

3 · var / let / const

Property	var	let	const
scope	function	block	block
before line	undefined	TDZ	TDZ
reassign	yes	yes	no
redeclare	usually	no	no
production	legacy	selective	default

Calling functions before their line

function f() {} callable
var f = () => {} TypeError
const f = () => {} ReferenceError

Shadowing ≠ redeclaration

Different scopes → different bindings.

Same scope → conflict / SyntaxError.

switch: one CaseBlock for all cases.

Use {} around each case body.

4 · Execution model

scope → where a name can be resolved

Environment Record → stores bindings

execution context → active state

call stack → unfinished calls

Call stack · LIFO

inner()

outer()

entry

A block {} may create a scope,
but not a new stack frame.

Lexical scope

Lookup follows where a function was
defined, not where it was called.

A closure keeps access to a binding,
not a frozen copy of its value.

5 · Common traps

typeof typeof absent → "undefined"
 typeof later; let later → ReferenceError

for-loop bindings

var: one shared binding

[3, 3, 3]

let: binding per iteration

[0, 1, 2]

const blocks reassignment, not object mutation.

delete removes a property, not a variable binding.

Early Error Invalid syntax stops evaluation;
even an earlier console.log will not run.

6 · Globals, strict mode & method

Where does a top-level name live?

classic script: var → globalThis property

classic script: let / const → global lexical

ES module: module-scoped + always strict

Node CommonJS: file has a function wrapper

Unresolved assignment

sloppy classic script → accidental global

strict / module → ReferenceError

Output-prediction method

1 host → 2 Early Errors → 3 scopes → 4 bindings

→ 5 lookup inner → outer → 6 execute → 7 classify

SyntaxError · ReferenceError · TypeError · undefined · value

30-second answer

Code is not moved: declarations are prepared before line-by-line execution.

var → undefined · let/const → TDZ · function declaration → ready · lookup goes inner → outer.